

本代码使用discretize管理离散化网格，将测量得到的磁场转化为磁铁的磁化强度

导入要用到的库

```
In [1]: import numpy as np
import pandas as pd
import glob
import os
from scipy.spatial import cKDTree
from scipy.sparse import kron, eye, csr_matrix, block_diag, vstack as sp_vstack
import discretize
from discretize import TreeMesh
import matplotlib.pyplot as plt
from scipy.sparse.linalg import lsqr
from scipy.optimize import minimize
from numba import njit, prange
```

配置参数

```
In [21]: # =====
# 1. 参数配置
# =====
csv_pattern = "specialU/*.csv"          # 多文件存放路径
output_vtk_base = "inverted_M_results_test"
B_unit_conversion = 1e-6

# 磁铁边界与几何（单位：mm）
MAGNET_X_MIN, MAGNET_X_MAX = 104, 186
MAGNET_Y_MIN, MAGNET_Y_MAX = 134, 240
MAGNET_Z_MIN, MAGNET_Z_MAX = -28, 0

X_c = (MAGNET_X_MIN + MAGNET_X_MAX) / 2.0
R_out = (MAGNET_X_MAX - MAGNET_X_MIN) / 2.0
Y_c = MAGNET_Y_MAX - R_out
R_in = 22

# ---- 自适应网格参数 ----
voxel_size_base = 1.5          # 基础网格尺寸
Y_split = 144.0                # 加密分界 y 坐标

# ---- 反演参数 ----
huber_epsilon = 4e3
max_iter = 4
lambda_reg = 1e-15
max_M = 1.5e6
```

读取磁场数据

多文件读取

磁场数据可以源于多个csv文件，将这些文件放到一个文件夹里，可以一起读取

Z镜像增强

我假设磁铁是镜像对称的，所以将磁场也镜像一份，这样可以更好收敛

```
In [22]: # =====
# 2. 多文件读取 + Z 镜像增强
# =====

print("正在读取测量数据...")
csv_files = glob.glob(csv_pattern)
if not csv_files:
    print("未找到数据")
    exit(1)
else:
    desired_file_list = [pd.read_csv(f) for f in csv_files]
    desired_file_combined = pd.concat(desired_file_list, ignore_index=True)

    measured_points_origin = desired_file_combined[['x','y','z']].values / 1000.0
    B_measured_origin = desired_file_combined[['Bx','By','Bz']].values * B_unit_conv

    Z_center_m = ((MAGNET_Z_MIN + MAGNET_Z_MAX)/2.0) / 1000.0
    measured_points_mirrored = measured_points_origin.copy()
    measured_points_mirrored[:,2] = 2*Z_center_m - measured_points_origin[:,2]
    B_measured_mirrored = B_measured_origin.copy()
    B_measured_mirrored[:,2] *= -1.0

    measured_points = np.vstack([measured_points_origin, measured_points_mirrored])
    B_measured = np.vstack([B_measured_origin, B_measured_mirrored])
    B_data = B_measured.ravel()
    # 序号变化（关键）：.ravel() 默认采用行优先（C-order）。
    # 原矩阵中第 $i$ 个点的分量为 $(B_{\{xi\}}, B_{\{yi\}}, B_{\{zi\}})$，
    # 展平后一维数组的索引排列为：$[B_{\{x0\}}, B_{\{y0\}}, B_{\{z0\}}, B_{\{x1\}}, B_{\{y1\}}, B_{\{z1\}}, \dots]$
    # 可以通过公式  $index = 3 * i + k$ （其中  $k \in \{0,1,2\}$  代表  $x, y, z$  分量）进行
    print(f"数据合并完成，共 {len(B_measured)} 个测点（含镜像）。")
    print("B_data 模长最大值:", np.max(np.abs(B_data)))
```

正在读取测量数据...

数据合并完成，共 8832 个测点（含镜像）。

B_data 模长最大值：0.0229803203

可变网格

这里需要注意，返回-1是最细的网格，也就是basemesh，返回0是最粗的网格，空间足够的话会一个格子填满所有 如果返回1就是完整边长分给一个格子 返回2就是两个 返回3就是四个 以此类推，数值越大越细

```
In [23]: # =====
# 3. 构建 TreeMesh 自适应网格
# =====

print("正在构建 TreeMesh 自适应网格...")

# 计算每个方向所需的2次幂基础单元数
nx_desired = int(np.ceil((MAGNET_X_MAX - MAGNET_X_MIN) / voxel_size_base))
```

```

ny_desired = int(np.ceil((MAGNET_Y_MAX - MAGNET_Y_MIN) / voxel_size_base))
nz_desired = int(np.ceil((MAGNET_Z_MAX - MAGNET_Z_MIN) / voxel_size_base))

def next_pow2(n):
    return 2 ** int(np.ceil(np.log2(n)))

nx_base = next_pow2(nx_desired)
ny_base = next_pow2(ny_desired)
nz_base = next_pow2(nz_desired)

# 计算基础网格实际单元尺寸（因为单元数取整后范围不变）
dx = (MAGNET_X_MAX - MAGNET_X_MIN) / nx_base
dy = (MAGNET_Y_MAX - MAGNET_Y_MIN) / ny_base
dz = (MAGNET_Z_MAX - MAGNET_Z_MIN) / nz_base

# 用(宽度, 数量)元组定义每个维度
hx = [(dx, nx_base)]
hy = [(dy, ny_base)]
hz = [(dz, nz_base)]

mesh = TreeMesh(
    [hx, hy, hz],
    origin=[MAGNET_X_MIN, MAGNET_Y_MIN, MAGNET_Z_MIN],
    diagonal_balance=False
)

# 定义显式的绝对细化层级，不再使用具有二义性的 -1
max_level = mesh.max_level
absolute_fine_level = max_level      # 最细一级（完全对应你设置的 voxel_size_base）
absolute_coarse_level = max_level - 1 # 次细一级（比最细的大 8 倍左右）

# 第一步：强行将磁铁的整体外包络框（Bounding Box）细化到基础层级
# 这样可以确保无论根节点怎么对齐，磁铁区域都绝对不会被“漏标”
BBox = np.array([
    [MAGNET_X_MIN, MAGNET_Y_MIN, MAGNET_Z_MIN], # 最小边界点
    [MAGNET_X_MAX, MAGNET_Y_MAX, MAGNET_Z_MAX]  # 最大边界点
])
mesh.refine_bounding_box(BBox, level=absolute_coarse_level)

print(f"基础网格实际单元尺寸：dx={dx:.2f}, dy={dy:.2f}, dz={dz:.2f} mm")
print(f"基础网格单元数：{nx_base}*{ny_base}*{nz_base}")

# 定义细化函数：根据 U 形几何和 y 坐标决定细化级别
def refine_func(cell):
    x, y, z = cell.center
    # 判断是否在磁铁内部
    if y > Y_c:
        dist = np.sqrt((x - X_c)**2 + (y - Y_c)**2)
        inside = (R_in <= dist <= R_out)
    else:
        inside = (MAGNET_Y_MIN <= y <= Y_c) and \
            ((X_c - R_out <= x <= X_c - R_in) or (X_c + R_in <= x <= X_c +
            if not inside:
                return 0 # 空气区不细化
    # 磁铁区根据 y 坐标细化
    return absolute_fine_level if y < Y_split else absolute_coarse_level

mesh.refine(refine_func)
mesh.number()
print(f"基础网格实际单元尺寸：dx={dx:.2f}, dy={dy:.2f}, dz={dz:.2f} mm | Max Level")

```

```

# 在jupyter notebook里无法交互，而且看起来很丑，于是注释掉了
# mesh.plot_grid(nodes=True)
# plt.show()

# 获取全体素中心和体积
all_cc = mesh.cell_centers          # mm
all_vol = mesh.cell_volumes        # mm³

# 判定磁铁掩膜
is_magnet = np.zeros(mesh.nC, dtype=bool)
for i, (x, y, z) in enumerate(all_cc):
    if y > Y_c:
        dist = np.sqrt((x - X_c)**2 + (y - Y_c)**2)
        is_magnet[i] = (R_in <= dist <= R_out)
    else:
        is_magnet[i] = (MAGNET_Y_MIN <= y <= Y_c) and \
            ((X_c - R_out <= x <= X_c - R_in) or (X_c + R_in <= x <=

magnet_idx = np.asarray(is_magnet).nonzero()[0]
# magnet_idx = np.where(is_magnet)[0]
voxel_points_mm = all_cc[magnet_idx]          # mm
voxel_points = voxel_points_mm / 1000.0        # m
voxel_vol = (all_vol[magnet_idx] / 1000**3)    # m³
n_voxels = len(voxel_points)

print(f"自适应网格：总单元 {mesh.nC}，磁铁单元 {n_voxels}")
print(f"最小单元尺寸约 {min(all_vol)**(1/3):.1f} mm，最大约 {max(all_vol)**(1/3)}

```

正在构建 TreeMesh 自适应网格...

基础网格实际单元尺寸：dx=1.28, dy=0.83, dz=0.88 mm

基础网格单元数：64*128*32

基础网格实际单元尺寸：dx=1.28, dy=0.83, dz=0.88 mm | Max Level: 6

自适应网格：总单元 32768，磁铁单元 15872

最小单元尺寸约 2.0 mm，最大约 2.0 mm

构建敏感度矩阵 A

$$B = A \cdot M$$

这里 B 展开是 $[B_{1x}, B_{1y}, B_{1z}, B_{2x}, B_{2y}, B_{2z}, \dots]$

M 展开是 $[M_{1x}, M_{1y}, M_{1z}, M_{2x}, M_{2y}, M_{2z}, \dots]$

A 是 $3M \times 3N$ 矩阵 这里大量使用了numpy的广播特性

$$\mathbf{B}(\mathbf{r}_i) = \frac{\mu_0 \Delta V_j}{4\pi} \left[\frac{3(\mathbf{M}_j \cdot \mathbf{dr}_{ij})\mathbf{dr}_{ij}}{r_{ij}^5} - \frac{\mathbf{M}_j}{r_{ij}^3} \right]$$

其中： $\mathbf{dr}_{ij} = \mathbf{r}_i - \mathbf{r}_j = (dx_{ij}, dy_{ij}, dz_{ij})^T$ 是从体素 j 指向测点 i 的位移矢量。

$r_{ij} = \|\mathbf{dr}_{ij}\|_2$ 是它们之间的欧氏距离。 $\mu_0 = 4\pi \times 1e^{-7}$ ，因此 $\frac{\mu_0}{4\pi} = 10^{-7}$ 。由于我

们要通过解线性方程组 $\mathbf{A}\mathbf{M} = \mathbf{b}$ 来反求 $\mathbf{M}$ ，我们需要把公式改写为对 $\mathbf{M}_j$ 的分量形式。例如，若仅看 M_{xj} 对测点 i 产生的三个磁场分量的贡献（此时

$\mathbf{M}_j = [M_{xj}, 0, 0]^T$ ），公式转化为：

$$\mathbf{B}_{contrib} = \frac{\mu_0 \Delta V_j}{4\pi} \left[\frac{3 \cdot dx_{ij} \cdot \mathbf{dr}_{ij}}{r_{ij}^5} - \frac{[1, 0, 0]^T}{r_{ij}^3} \right] \cdot M_{xj}$$

```
In [24]: # =====
# 4. 构建敏感度矩阵 A (偶极子正演)
# =====
print("正在计算敏感度矩阵...")
def build_matrix(meas, voxels, vols):
    M, N = len(meas), len(voxels)
    A = np.zeros((M*3, N*3))
    mu0_4pi = 1e-7
    coef = (mu0_4pi * vols)[None, :, None] # 广播体积权重, 这里第一维和第三维由上

    dr = meas[:, None, :] - voxels[None, :, :] # 结果dr是(M, N, 3),
    r = np.linalg.norm(dr, axis=2)
    r = np.maximum(r, 1e-4)
    r3, r5 = r**3, r**5
    dx, dy, dz = dr[:, :, 0], dr[:, :, 1], dr[:, :, 2] # 这里的都是(M, N)

    # 下面计算第col个M分量产生的B贡献, 然后填充到A中, term1、term2都是(M, N, 3)的
    for col, d_comp in enumerate([dx, dy, dz]):
        term1 = 3 * d_comp[:, :, None] * dr / r5[:, :, None]
        eye = np.zeros(3)
        eye[col] = 1.0
        term2 = eye[None, None, :] / r3[:, :, None]
        B_contrib = coef * (term1 - term2)
        for k in range(3):
            A[k::3, col::3] = B_contrib[:, :, k] # k::3是切片, start:stop:step, 填充
    return A

@njit(parallel=True) # 开启多线程并行
def build_matrix_numba(meas, voxels, vols):
    M = meas.shape[0]
    N = voxels.shape[0]
    # 预分配最终的 A 矩阵, 这部分内存是省不掉的
    A = np.zeros((M * 3, N * 3))
    mu0_4pi = 1e-7

    # 使用 prange 进行并行化循环
    for i in prange(M):
        for j in range(N):
            # 直接计算, 不生成 (M, N, 3) 的大矩阵
            dx = meas[i, 0] - voxels[j, 0]
            dy = meas[i, 1] - voxels[j, 1]
            dz = meas[i, 2] - voxels[j, 2]

            r2 = dx*dx + dy*dy + dz*dz
            if r2 < 1e-8: # 对应你原代码中的 1e-4
                r2 = 1e-8

            r = np.sqrt(r2)
            r3 = r2 * r
            r5 = r3 * r2

            coef = mu0_4pi * vols[j]
            dr = (dx, dy, dz)

            # 计算 3x3 的偶极子耦合块
```

```

        for col in range(3):
            d_comp = dr[col]
            for k in range(3):
                term1 = 3.0 * d_comp * dr[k] / r5
                term2 = 1.0 / r3 if col == k else 0.0

                val = coef * (term1 - term2)
                # 映射回全局 A 矩阵
                A[3 * i + k, 3 * j + col] = val

    return A

# 调用方式完全一样
A = build_matrix_numba(measured_points, voxel_points, voxel_vol)
# A = build_matrix(measured_points, voxel_points, voxel_vol)

```

正在计算敏感度矩阵...

构建正则化矩阵

由于单纯反演的结果非常混乱，我需要加入一些人为限制帮助收敛，这里我认为磁化强度不可以突变，所以将磁化强度的梯度作为惩罚项。

但是又要突出接缝，所以当梯度大于一个阈值时，减小惩罚项的权重。

这里的矩阵 G 用于计算梯度，原始的 G 通过`mesh.cell_gradient`得到，将不需要的部分剔除

```

In [25]: # =====
# 5. 利用 cell_gradient 构建正则化矩阵
# =====
print("正在构建正则化矩阵（基于 TreeMesh.cell_gradient）...")

# 【安全防护】确保 G_full 显式转换为 CSR 格式
# 因为某些版本的 discretize 算子可能是通用稀疏矩阵，不直接暴露 .indptr 和 .indices
G_full = mesh.cell_gradient
if not isinstance(G_full, csr_matrix):
    G_full = G_full.tocsr()

row_start = G_full.indptr          # (行指针) 记录每一行非零元素在数据区中的起始和结束
col_indices = G_full.indices       # (列索引) 紧凑存储所有非零元素对应的全局单元序号
n_faces = G_full.shape[0]

# 过滤属于磁铁内部的有效约束面
internal_faces = []
for face_idx in range(n_faces):
    cols = col_indices[row_start[face_idx]:row_start[face_idx+1]]
    if len(cols) >= 2:              # 内部面（含悬挂面）
        if is_magnet[cols].all():  # 所有相关相邻单元都在磁铁实体内部
            internal_faces.append(face_idx)

internal_faces = np.array(internal_faces)
n_valid_faces = len(internal_faces)
print(f"内部约束面数量（含悬挂面）：{n_valid_faces}")

# 裁剪矩阵：行方向只保留内部面，列方向只保留属于反演区域的磁铁单元
G_sub = G_full[internal_faces, :][:, magnet_idx]

```

```
# 利用张量积 (Kronecker Product) 膨胀算子
# 将 G_sub (n_valid_faces, n_voxels) 中的每个差分系数元素都膨胀为一个 3x3 的单位阵
# 这样不仅维持了三个分量的解耦差分, 同时令列布局主动契合 [M1x, M1y, M1z, M2x, M2y, M2z]
G_big = kron(G_sub, eye(3, format="csr"))
```

正在构建正则化矩阵 (基于 TreeMesh.cell_gradient) ...

内部约束面数量 (含悬挂面): 44240

求解

数学原理: 此处采用的是类 Huber 惩罚函数 (也称 Charbonnier 损失), 用于在数学上平滑近似标准 Huber 范数:

$$\phi_m = \lambda \sum_f \epsilon^2 \left(\sqrt{1 + \left(\frac{\|\nabla \mathbf{M}\|_f}{\epsilon} \right)^2} - 1 \right)$$

当梯度小 ($\ll \epsilon$) 时, 它退化为平方平滑约束 (L_2 范数); 当梯度大 ($\gg \epsilon$) 时, 它退化为线性边缘保持约束 (L_1 范数), 从而允许磁铁接缝处存在突变, 不会把边界模糊掉。

```
In [26]: # =====
# 6. 基于 L-BFGS-B 的高效边缘保持反演
# =====
print("开始 L-BFGS-B 高效反演...")

# 1. 准备常数与初始解 (一维展平向量)
M_init = np.zeros(3 * n_voxels)

# 2. 定义严格的物理边界约束 [-max_M, max_M]
bounds = [(-max_M, max_M)] * (3 * n_voxels)

# 3. 构建目标函数及其梯度计算 (带等比例缩放)
def objective_and_gradient(M_vec):
    M_sol_current = M_vec.reshape(n_voxels, 3)

    # ---- 1. 数据拟合项 (Data Misfit) ----
    residual = A @ M_vec - B_data
    loss_data = np.sum(residual**2)
    grad_data = 2.0 * (A.T @ residual)

    # ---- 2. Huber 正则化项 (Huber Regularization) ----
    grad_x = G_sub @ M_sol_current[:, 0]
    grad_y = G_sub @ M_sol_current[:, 1]
    grad_z = G_sub @ M_sol_current[:, 2]

    grad_norm = np.sqrt(grad_x**2 + grad_y**2 + grad_z**2)
    loss_reg = lambda_reg * huber_epsilon**2 * np.sum(np.sqrt(1.0 + (grad_norm /
    weights_current = 1.0 / np.sqrt(1.0 + (grad_norm / huber_epsilon)**2)
    g_reg_x = G_sub.T @ (grad_x * weights_current)
    g_reg_y = G_sub.T @ (grad_y * weights_current)
    g_reg_z = G_sub.T @ (grad_z * weights_current)
    grad_reg = lambda_reg * np.column_stack([g_reg_x, g_reg_y, g_reg_z]).ravel()
```

```

# ---- 3. 汇总并进行 ----
total_loss = loss_data + loss_reg
total_grad = grad_data + grad_reg

return total_loss, total_grad

# 4. 调用高级拟牛顿求解器
# 【关键点 2】把 ftol 和 gtol 设得极小，配合前面的缩放，彻底粉碎“提前终止”
res_opt = minimize(
    fun=objective_and_gradient,
    x0=M_init,
    jac=True,
    method='L-BFGS-B',
    bounds=bounds,
    options={
        'maxiter': 50,
        'disp': True,
        'ftol': 1e-30, # 允许能量函数发生微小的变化
        'gtol': 1e-30 # 允许梯度无限变小
    }
)

# 5. 还原最终解
M_sol = res_opt.x.reshape(n_voxels, 3)
print(f"L-BFGS-B 反演完成！最大反演磁化强度：{np.max(np.abs(M_sol)):.3e} A/m")

```

开始 L-BFGS-B 高效反演...

/tmp/ipykernel_101168/2405946628.py:43: DeprecationWarning: scipy.optimize: The `disp` and `iprint` options of the L-BFGS-B solver are deprecated and will be removed in SciPy 1.18.0.

```
res_opt = minimize(
```

L-BFGS-B 反演完成！最大反演磁化强度：3.125e+05 A/m

导出

直接导出自适应网格至 VTK 文件

```

In [27]: # =====
# 7. 直接导出自适应网格至 VTK 文件（免去重新投影）
# =====
print("正在将反演结果映射回原始自适应网格...")

# 1. 创建一个与原自适应网格总单元数 (mesh.nC) 一致的全零矩阵，包含 3 个空间分量
# 这样空气区域的磁化强度将默认保持为 0（这在物理上是完全正确的）
M_full = np.zeros((mesh.nC, 3))

# 2. 利用之前记录的磁铁索引 magnet_idx，将反演结果 M_sol (n_voxels, 3) 精准填回为
M_full[magnet_idx, :] = M_sol

# 3. 构造导出模型字典
# 既包含 3D 矢量场，也包含拆分后的单轴标量场，极大地方便在 ParaView 中灵活切换和
model_dict = {
    "M_vector": M_full, # 3D 矢量场（用于在 ParaView 中绘制箭头或计算总
    "Mx": M_full[:, 0], # X 方向标量磁化强度
    "My": M_full[:, 1], # Y 方向标量磁化强度
    "Mz": M_full[:, 2]  # Z 方向标量磁化强度
}

```

```
# 4. 直接调用 TreeMesh 自带的 write_vtk 方法
# 注意: 不需要写后缀名, discretize 会根据网格类型自动生成 "文件名.vtu"
print(f"正在直接写入 VTK (UnstructuredGrid) 文件: {output_vtk_base}.vtu ...")

mesh.write_vtk(output_vtk_base, models=model_dict)

print(f"✅ VTK 导出成功! 文件已写入 {output_vtk_base}.vtu")
```

正在将反演结果映射回原始自适应网格...

正在直接写入 VTK (UnstructuredGrid) 文件: inverted_M_results_test.vtu ...

✅ VTK 导出成功! 文件已写入 inverted_M_results_test.vtu